Lab 13 — Questions

Question 1 [Diagnosis]

```
$ kubectl get pods
NAME                   READY   STATUS             RESTARTS      AGE
api-6c7bb54fc5-2vs4k   1/1     Running            0             27s
web-5588bd85f8-j5nl5   0/1     CrashLoopBackOff   2 (6s ago)    27s
$ kubectl logs deploy/web --tail=1
nginx: [emerg] host not found in upstream "api" in /etc/nginx/conf.d/default.conf:15
```

The API is running. Why can nginx not find `api`, when the same configuration worked in lab 07 with a plain Podman network? What must be created, and why does `web` then recover on its own?

Question 2 [Understanding]

Explain what, for an `api` Service, the **cluster IP**, the **DNS name** and the **EndpointSlices** are, who maintains each, and why `ping 10.43.3.7` never answers while `wget http://10.43.3.7:8080/` works.

Question 3 [Prediction]

An `api` Service selects `app: api`; the Deployment goes from 2 to 3 replicas, then you delete a Pod by hand, then you break the readiness of one Pod. What does `kubectl get endpointslices` show after each operation, and what did a client calling `api:8080` see in the meantime?

Question 4 [Diagnosis]

`kubectl get svc api` shows a ClusterIP and the `8080/TCP` port, but `kubectl get endpointslices -l kubernetes.io/service-name=api` returns a slice **with no address**, and Traefik answers `503`. The Pods are `1/1 Running`. Name the two possible causes, the command that tells them apart, and the fix for each.

Question 5 [Analysis]

Compare `ClusterIP`, `NodePort` and `LoadBalancer` for exposing the front end on your WSL: which URL in each case, what k3s does beyond bare Kubernetes for the third one, and why a second `LoadBalancer` Service on port 80 would stay `<pending>` with k3s.

Question 6 [Understanding]

From a Pod in the `lab13` namespace, `wget http://api:8080/` works; from a Pod in the `default` namespace, it fails. Explain with the content of a Pod's `/etc/resolv.conf`, and give the name to use from `default`.

Question 7 [Analysis]

A Service spreads connections, not requests. Explain the difference, what it means for an HTTP/1.1 client with *keep-alive* (or a JDBC connection) facing three replicas, and what the Ingress changes about that behaviour.

Question 8 [Diagnosis]

```
spec:
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 8080
```

With this Service, `wget http://api:8080/` fails (`Connection refused`) but `wget http://api/` works. Explain the two ports, and say what nginx must write in its `proxy_pass`.

Question 9 [Analysis]

Describe the path of a `GET /api/actuator/health` request sent by a Windows browser to `http://localhost/` in our k3s stack: every component crossed, in order, and the two reverse proxies involved. How can the chain be simplified by moving one rule?

Question 10 [Understanding]

What is the difference between an `Ingress` object and an *Ingress controller*? What happens if you apply an Ingress in a cluster with no controller? And why does a `503 Service Unavailable` returned by Traefik almost always point to an endpoints problem rather than a Traefik problem?

Question 11 [Diagnosis]

A team deploys its application on EKS with the manifests written on k3s. The `LoadBalancer` Service does get a public IP, but the Ingress stays without an `ADDRESS` and the site does not answer. What is missing, and why did it "work" on k3s?

Question 12 [Analysis]

The front end calls the API through `/api/` (same origin) rather than through `http://api.example.com` (different origin). Recall why (lab 07), then say where that `/api/` routing can be done in our stack — nginx or Ingress — and the benefits of entrusting it to the Ingress.

Question 13 [Analysis]

A developer wants to reach the cluster's PostgreSQL database from DBeaver on Windows. Three options: `kubectl port-forward svc/db 5432:5432`, a NodePort, a LoadBalancer. Rank them by increasing exposure, say which one is acceptable in production and why, and which object is still missing for `db` to have a name (hint: lab 15).